Strategy Pattern + OCP
Easy Notes for LinkedIn and LLD

Day 5 - My System Design / LLD Journey

1) LinkedIn Post Draft

🚀 Day 5 of My System Design Journey

Today I understood one of the most important ideas in LLD:

Strategy Pattern is not just a design pattern. It is a response to real software pain.

I used to think payment code is simple: if UPI then do this, if Card then do that. But as soon as more payment methods arrive, the code starts growing, testing becomes harder, and every small change feels risky.

That is exactly why Strategy Pattern exists.

Instead of putting all logic inside one big if-else block, we split each payment method into its own class. Then the main service only asks the selected strategy to do the job.

That is the real shift:

"Do everything myself" -> "Delegate to the right class"

What I learned today:

- OCP says: do not keep modifying tested code.
- Strategy Pattern helps us follow OCP in a clean way.
- Each payment method becomes independent and easy to test.
- New behavior can be added with minimal changes.

Example: UPI, Card, Wallet, Net Banking, Crypto - each one can be a separate strategy.

This is why recruiters and interviewers ask about Strategy Pattern: not because they want a definition, but because they want to see whether you can design code that survives change.

If you are also learning LLD, join me on this journey. I am sharing simple notes, examples, and interview-friendly explanations every day.

Comment "LLD" and connect with me. Let us grow together. 🚀

#SystemDesign #LowLevelDesign #LLD #StrategyPattern #DesignPatterns #SOLID #OCP #Java #Python #SoftwareEngineering #LearningInPublic

2) The Real Problem

Suppose you are building a payment system.

- Day 1: only Card payment exists.
- Month 3: UPI is added.
- Month 5: Wallet is added.
- Month 7: Crypto is added.

If all of them live inside one big if-else block, the file keeps growing.

That becomes hard to read, hard to test, and risky to change.

3) Before OCP

Before OCP, the code often looks like this:

```
class PaymentService {
   public void pay(String type) {
      if(type.equals("UPI")) {
         System.out.println("UPI payment");
      } else if(type.equals("CARD")) {
         System.out.println("Card payment");
      } else if(type.equals("WALLET")) {
         System.out.println("Wallet payment");
      }
   }
}
```

Problem: every new payment method requires editing the same class.

4) OCP Explained Simply

Open-Closed Principle means:

- Open for extension
- Closed for modification

In simple words: add new features without touching old, stable code.

OCP is the rule. It tells us what good design should look like.

5) Why OCP Alone Was Not Enough

Even if we split Card, UPI, and Wallet into different classes, we still need a clean way to choose which one to run at runtime.

Otherwise the if-else may just move to another file.

So the problem is not only 'how to create classes'.

The bigger problem is 'how to switch behavior cleanly at runtime'.

6) Strategy Pattern

Strategy Pattern solves exactly this.

- Create one common interface.
- Put each behavior in its own class.
- Inject the correct behavior from outside.
- The main class uses the behavior without knowing the details.

7) Easy Real-World Example

Think about a food app.

- One user pays by UPI.
- Another user pays by Card.
- Another user pays by Wallet.
- Tomorrow a new method comes: PayPal.

You do not want to rewrite the full payment flow every time.

You just want to add one new class and leave the old code untouched.

8) Java Example

```
abstract class Payment {
   abstract void pay();
}

class UpiPayment extends Payment {
   void pay() {
      System.out.println("UPI");
   }
```

```
}

class CardPayment extends Payment {
   void pay() {
      System.out.println("CARD");
   }
}

class WalletPayment extends Payment {
   void pay() {
      System.out.println("WALLET");
   }
}
```

Now each payment type is a separate class. One class = one job.

9) How Strategy Works in Practice

A service does not care whether the payment is UPI or Card.

It only knows that some object can pay.

```
interface PaymentStrategy {
   void pay();
}

class PaymentService {
   private PaymentStrategy strategy;

   public PaymentService(PaymentStrategy strategy) {
      this.strategy = strategy;
   }

   public void processPayment() {
      strategy.pay();
   }
}
```

Usage:

```
PaymentService service = new PaymentService(new UpiPayment());
service.processPayment();
```

10) Why This Is Better

- No giant if-else chain in the core service
- Easy to add new payment types
- Easy to test one strategy at a time
- Less risk when changing one behavior
- Cleaner code and better maintainability

11) One-Line Mental Model

Without Strategy Pattern: 'I will handle every payment myself.'

With Strategy Pattern: 'I will ask the right helper to do the payment.'

12) OCP vs Strategy

OCP is the principle.

Strategy Pattern is one way to achieve that principle in code.

- OCP = what should happen
- Strategy = how we implement it

13) Interview Answer

Strategy Pattern is used when we have multiple interchangeable behaviors and we want to select one at runtime without changing the main business logic. It helps us follow the Open-Closed Principle and reduces large if-else blocks.

14) Invite for the Journey

I am also building my LLD notes in a simple way.

If someone wants to learn together, I will keep sharing:

- Easy notes
- Java examples
- Python examples
- Interview questions
- Real-world design patterns

Comment 'LLD' and connect with me on LinkedIn.